

LYCANS AERODESIGN TEAM

Training Task 05

ROS2 & Simulation

Deadline: Aug 30, 2026 - 11:59 PM

Submission: GitHub Repository & Google Drive Link

1 Objective

In this assignment, you will build a 3-node ROS 2 telemetry processing and autonomous decision system, as well as simulate an automated mission using Gazebo and ArduPilot SITL.

2 Part 1 — Building a Basic ROS 2 Telemetry System

2.1 Objective

Design and implement a multi-node ROS 2 system that simulates aircraft telemetry streaming, processes parsed data to make rule-based failsafe decisions, and commands mode updates to an actuator node.

You will practice:

- How to build a ROS 2 system from scratch
- Data serialization & parsing
- Rule-based decision logic
- Debugging and tracing inter-node communication

2.2 Node Specifications

2.2.1 Node 1: Telemetry Generator

- Simulating telemetry data by Periodically generating random telemetry values within these ranges:
 - Altitude: 30 m to 80 m
 - Airspeed: 12 m/s to 24 m/s
 - Voltage: 12 V to 30 V
 - GPS Fix: True or False (boolean)
- Packs the data into a single ordered, delimiter-separated string format: "altitude,airspeed,battery_voltage,gps_fix" (e.g., "42.5,14.2,12.4,True").
- Publishes this string to a topic

2.2.2 Node 2: Decision Maker

- Subscribes to the raw telemetry string and parses it back into numerical/boolean variables.
- Evaluates the values using the following priority failsafe rules:
 - if battery_voltage < 13.0 → RTL
 - Else if gps_fix == False → RTL
 - Else if altitude < 50.0 → MANUAL
 - Else if airspeed < 16.0 → FBWA
 - Else (nominal conditions) → AUTO
- Publishes the determined mode string to a decision topic

2.2.3 Node 3: Mode Actuator

- Subscribes to the flight mode decision topic.
- Simulates vehicle mode switching by logging directly to the console:
"Flight mode changed to <MODE>"

3 Part 2 — Gazebo Simulation & Mission Execution

3.1 Objective

In this part, you will simulate the exact same mission you created in the previous task using Gazebo.

3.2 Simulation Requirements

- Ensure Gazebo and the necessary ArduPilot plugins are successfully installed.
- Download and set up the **Skywalker** aircraft model for Gazebo.
- Connect the Skywalker model to ArduPilot SITL and **QGroundControl (QGC)**.
- Execute and simulate your previous mission within the Gazebo environment, monitoring its progress via QGroundControl.

Deliverables & Submission Guidelines

Submit your work through a GitHub repository containing the following items:

1. Video Demonstration

Record a clear screen recording demonstrating your working ROS 2 system and your Gazebo simulation. Upload the video to a **Google Drive** folder and ensure the sharing permissions are set to **“Anyone with the link can view”**.

Your video **MUST** include:

- **Voice Explanation:** A brief overview explaining how your code works **with your own voice** while demonstrating execution.
- **Live Node Execution:** Clear view of all three running terminals displaying real-time node logs.
- **Topic Inspection:** Running `ros2 topic echo <topic_name>` for each active topic in the pipeline to verify data transmission.
- **Computation Graph:** The active `rqt_graph` showing all three nodes and their connecting topics.
- **Gazebo & QGC Simulation:** A clear view of Gazebo running the mission with the Skywalker model while connected to and monitored on QGroundControl.

2. GitHub Repository Contents

Your submitted repository must contain:

- The Python source code files for all three nodes (.py files).
- A .txt file containing the Google Drive link to your video recording (verified accessible via “Anyone with the link”).

ROS2 Node Code Template

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MyCustomNode(Node):
    def __init__(self):
        super().__init__("my_custom_node")
        self.get_logger().info("Message printed to console")

def main(args=None):
    rclpy.init(args=args)          # Initialize ROS 2
    node = MyCustomNode()          # Create an instance of your node
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```